

DESARROLLO DE SOFTWARE

Clase V: Paradigmas de Diseño

SOLID, patrones de diseño y buenas prácticas del código

2026



PRINCIPIOS SOLID

SOLID es un acrónimo de 5 principios de diseño orientado a objetos que hacen el software más mantenible, flexible y escalable. Introducidos por Robert C. Martin (Uncle Bob).

S

Single Responsibility
Responsabilidad única

Una clase debe tener una sola razón para cambiar.

O

Open / Closed
Abierto –Cerrado

Abierto para extensión, cerrado para modificación.

L

Liskov Substitution
Sustitución de Liskov

Las subclases deben ser sustituibles por sus clases base.

I

Interface Segregation
Segregación de interfaces

Interfaces específicas mejor que una general y grande.

D

Dependency Inversion
Inversión de dependencias

Dependé de abstracciones, no de implementaciones concretas.

PRINCIPIOS SOLID – EJEMPLOS APLICABLES

S

(Single Responsibility): Una clase solo debe hacer una cosa.

Cómo aplicarlo: Si tenés una clase *Factura* que calcula el total y también la guarda en la base de datos, sepárala. Creá una clase *Factura* y otra *FacturaRepository* para el guardado.

O

(Open/Closed): El código debe estar abierto a extensión pero cerrado a modificación.

Cómo aplicarlo: Si necesitas agregar un nuevo método de pago (ej. Cripto), no modifiques tu función principal de pagos con mil if/else. Usa una interfaz y agrega una nueva clase para Cripto.

L

(Liskov Substitution): Una clase hija debe poder usarse en lugar de su clase padre sin romper nada.

Cómo aplicarlo: Si tenés una clase *Ave* con el método *volar()*, no crees una subclase *Pinguino* que herede de ella, porque los pingüinos no vuelan y romperías la lógica.

I

(Interface Segregation): Mejor muchas interfaces chiquitas que una gigante.

Cómo aplicarlo: No obligues a una clase a implementar métodos que no usa. Si tenés una interfaz *AccionesTrabajador*, sepárala en *TrabajadorPresencial* y *TrabajadorRemoto* para que el remoto no tenga que implementar un método *ficharEntradaOficina*.

D

(Dependency Inversion): Depende de abstracciones (interfaces), no de cosas concretas.

Cómo aplicarlo: Tu lógica de negocio no debería depender de si usás MySQL o MongoDB. Debería depender de una interfaz *BaseDeDatos*.

PATRONES DE DISEÑO — CREACIONALES

¿QUÉ SON LOS PATRONES DE DISEÑO?

Estos patrones proporcionan un enfoque estructurado y reutilizable para resolver situaciones recurrentes en el desarrollo de software.

Cada patrón de diseño tiene un propósito específico y ofrece una solución a un problema de diseño común.

Utilizar los patrones resulta en un software más eficiente, mantenible y escalable.

Es importante tener en cuenta que los patrones de diseño no son algoritmos o código listo para usar, sino más bien pautas y descripciones de soluciones de diseño.

Los desarrolladores deben adaptar e implementar los patrones de diseño de acuerdo con los requisitos y características específicas de su proyecto de software.

Son técnicas para resolver problemas recurrentes de diseño de software.



COMPORTAMENTALES

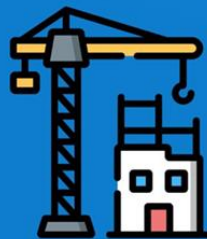
Establece soluciones relacionados con el comportamiento de la aplicación respecto a la interacción entre objetos y clases.



Ej. Observer, State.

CREACIONALES

Soluciona la creación de objetos, hace un sistema independiente de cómo sus objetos son creados.



Ej. Singleton, Factory.

ESTRUCTURALES

Describe cómo los objetos se componen para formar estructuras complejas.



Ej. Adapter, Decorator.

PATRONES DE DISEÑO — CREACIONALES

Los patrones de diseño son soluciones reutilizables a problemas comunes de diseño de software. Propuestos por la 'Gang of Four' en 1994.

Singleton

Garantiza que una clase tenga solo UNA instancia. Uso: conexión a base de datos, configuración global.

Factory Method

Define una interfaz para crear objetos, pero deja a las subclasses decidir cuál clase instanciar.

Abstract Factory

Crea familias de objetos relacionados sin especificar sus clases concretas.

Builder

Construye objetos complejos paso a paso. Separa la construcción de la representación.

Prototype

Crea nuevos objetos copiando (clonando) un objeto existente.

Object Pool

Reutiliza objetos costosos de crear. Uso: conexiones de DB, hilos de ejecución.

PATRONES ESTRUCTURALES Y DE COMPORTAMIENTO

ESTRUCTURALES

Adapter

Convierte la interfaz de una clase en otra que el cliente espera.

Decorator

Agrega responsabilidades a un objeto dinámicamente.

Facade

Proporciona una interfaz simplificada a un sistema complejo.

Composite

Compone objetos en estructuras de árbol para representar jerarquías.

Proxy

Proporciona un sustituto o marcador de posición para otro objeto.

COMPORTAMIENTO

Observer

Notifica automáticamente a dependientes cuando un objeto cambia.

Strategy

Define familia de algoritmos intercambiables en tiempo de ejecución.

Command

Encapsula una acción como objeto, permitiendo deshacer/rehacer.

Iterator

Proporciona acceso secuencial a elementos sin exponer la estructura.

Template Method

Define el esqueleto de un algoritmo, delegando pasos a subclases.

PATRONES DE DISEÑO — EJEMPLOS APLICABLES

Creacionales (Cómo crear objetos)

- **Singleton:** Úsalo cuando necesites **una sola instancia** en toda la app (ej. la configuración del sistema o la conexión a la DB).
- EJ: Un sistema de **Log de errores**. No quieres 10 archivos de log abiertos al mismo tiempo; usas una sola instancia global para que toda la app escriba en el mismo lugar.
- **Factory Method:** En lugar de hacer `new Objeto()`, llamas a una "fábrica" que decide qué objeto darte según la situación.
- EJ. Un sistema de **Logins sociales**. Según el botón que toque el usuario (Google, Facebook o GitHub), la "fábrica" te devuelve el objeto de autenticación correcto.

Estructurales (Cómo organizar clases)

- **Adapter:** Como un adaptador de viaje. Sirve para que dos clases con interfaces incompatibles puedan trabajar juntas.
- EJ. Quieres usar una librería moderna de mapas que usa coordenadas `(lat, long)`, pero tu base de datos vieja guarda los datos en formato `(x, y)`. Creas un **adaptador** que convierta los datos viejos al formato nuevo
- **Decorator:** Para añadir funciones a un objeto sin cambiar su estructura
- EJ. Un sistema de **notificaciones**. Tienes el envío básico por Email, pero puedes "decorarlo" para que también envíe por SMS, y luego otro "decorador" para WhatsApp, sin cambiar la clase original.

Comportamiento (Cómo se comunican)

Observer: Muy común en interfaces visuales. Un objeto avisa a otros cuando cambia algo (ej. cuando llega un mensaje nuevo, se actualiza el icono de notificación). EJ. El sistema de **carrito de compras**. Cuando el stock de un producto llega a cero, el objeto "Producto" le avisa a todos los usuarios que lo tenían en su "Lista de deseos" para que vean el cartel de "Sin stock".

Strategy: Te permite cambiar el algoritmo que usás en tiempo de ejecución (ej. elegir entre pagar con Tarjeta o PayPal según lo que elija el usuario).

EJ. Una aplicación de **mapas (GPS)**. Según si eliges ir "Caminando", "En Auto" o "En Bici", el sistema cambia el algoritmo de cálculo de ruta dinámicamente.

IA Y PATRONES DE DISEÑO — 2026

La IA puede reconocer, sugerir y aplicar patrones de diseño en tu código, además de identificar antipatrones (code smells) automáticamente.

◆ Detección de patrones

GitHub Copilot y Claude pueden identificar qué patrón de diseño es más apropiado para un problema dado, con explicación incluida.

◆ Refactoring inteligente

Herramientas como SonarQube + IA detectan code smells (código duplicado, clases demasiado largas) y sugieren patrones para resolver.

◆ Generación de arquitecturas

Prompt: 'Diseña una arquitectura para un sistema de notificaciones en tiempo real'. La IA sugiere patrones como Observer + Queue.

◆ IA como arquitecto asistente

Claude puede revisar diagramas UML o pseudocódigo y sugerir si el diseño viola principios SOLID o puede mejorarse con un patrón.

¡MUCHAS GRACIAS!

Clase V: Paradigmas de Diseño

Próxima clase: consultar cronograma.